

Selection of Tests for running in Playwright

In Playwright, test selection can be configured centrally in `playwright.config.ts` using several options depending on your needs.

1. Select Tests by Tag (grep)

Tests

```
import { test } from '@playwright/test';
```

```
test('Login @smoke', async ({ page }) => {  
  // test steps  
});
```

```
test('Checkout @regression', async ({ page }) => {  
  // test steps  
});
```

`playwright.config.ts`

```
import { defineConfig } from '@playwright/test';
```

```
export default defineConfig({  
  grep: /@smoke/  
});
```

Only tests containing `@smoke` will run.

2. Exclude Tests (grepInvert)

```
export default defineConfig({  
  grepInvert: /@wip/  
});
```

Tests tagged `@wip` will be skipped.

3. Run Specific Test Files (testMatch)

Suppose your structure is:

tests/

└─ login.spec.ts

└─ checkout.spec.ts

└─ search.spec.ts

playwright.config.ts

```
export default defineConfig({  
  testMatch: [  
    '**/login.spec.ts',  
    '**/checkout.spec.ts'  
  ]  
});
```

Only those files run.

4. Exclude Specific Files (testIgnore)

```
export default defineConfig({  
  testIgnore: [  
    '**/search.spec.ts'  
  ]  
});
```

Everything except search.spec.ts runs.

5. Run Tests from a Folder

Project structure:

tests/

└─ smoke/

| └─ login.spec.ts

```
| └─ search.spec.ts
└─ regression/
    └─ checkout.spec.ts
```

playwright.config.ts

```
export default defineConfig({
  testDir: './tests/smoke'
});
```

Only tests in the smoke folder run.

6. Configure Multiple Suites Using Projects

This is the most common enterprise approach.

Tests

```
test('Login @smoke', async () => {});
test('Checkout @regression', async () => {});
test('Search @sanity', async () => {});
```

playwright.config.ts

```
import { defineConfig } from '@playwright/test';
```

```
export default defineConfig({
  projects: [
    {
      name: 'smoke',
      grep: /@smoke/
    },
    {
      name: 'regression',
      grep: /@regression/
    },
  ],
});
```

```
{  
  name: 'sanity',  
  grep: /@sanity/  
}  
]  
});
```

Run:

```
npx playwright test --project=smoke
```

or

```
npx playwright test --project=regression
```

7. Environment-Based Selection

playwright.config.ts

```
import { defineConfig } from '@playwright/test';
```

```
const suite = process.env.SUITE;
```

```
export default defineConfig({
```

```
  grep:
```

```
    suite === 'smoke'
```

```
      ? /@smoke/
```

```
      : suite === 'regression'
```

```
      ? /@regression/
```

```
      : undefined
```

```
});
```

Execute

Linux/Mac:

```
SUITE=smoke npx playwright test
```

Windows CMD:

```
set SUITE=smoke && npx playwright test
```

PowerShell:

```
$env:SUITE="smoke"
```

```
npx playwright test
```

8. Select by Test Title

```
export default defineConfig({
```

```
  grep: /Valid Login/
```

```
});
```

Only tests whose titles contain "Valid Login" will run.

Real-World Framework Example

```
import { defineConfig } from '@playwright/test';
```

```
export default defineConfig({
```

```
  testDir: './tests',
```

```
  projects: [
```

```
    {
```

```
      name: 'smoke',
```

```
      grep: /@smoke/
```

```
    },
```

```
    {
```

```
      name: 'sanity',
```

```
      grep: /@sanity/
```

```
    },
```

```
    {
```

```
    name: 'regression',  
    grep: /@regression/  
  }  
],
```

```
  retries: 1,  
  workers: 4,
```

```
  use: {  
    headless: true  
  }  
});
```

Example test:

```
import { test } from '@playwright/test';
```

```
test('User Login @smoke @sanity', async ({ page }) => {  
  // test code  
});
```

```
test('Place Order @regression', async ({ page }) => {  
  // test code  
});
```

This setup allows you to run different suites from CI/CD pipelines simply by choosing the project:

```
npx playwright test --project=smoke
```

```
npx playwright test --project=sanity
```

```
npx playwright test --project=regression
```

This is typically the cleanest and most scalable way to select tests in a Playwright TypeScript framework.

